

Project 3: Reorder Buffer Extension to a 2-Wide Superscalar Tomasulo CPU

Sangeeth Thayaaparan
RIN: 662037988

April 21, 2026

Tools: ModelSim – Intel FPGA Starter Edition 2020.1, SystemVerilog RTL

Abstract

This report documents the Project 3 extension of my previously-submitted 2-wide superscalar Tomasulo CPU. The baseline design — in-order dispatch, three reservation stations, pipelined functional units, and tag-based forwarding over a Common Data Bus (CDB) — is retained. On top of it I added a 16-entry Reorder Buffer (ROB) that guarantees in-order *commit* of instructions that are dispatched in order but execute and write their results back out of order. The ROB entry index now doubles as the rename tag used throughout the pipeline, so the existing CDB and RS snoop logic required no semantic change — only a widening of the tag field from 3 to 4 bits. The extension adds (i) an ROB module with in-order commit, (ii) predict-not-taken branch handling with single-cycle blanket flush on misprediction, (iii) a one-entry store buffer that forwards stores-in-flight to younger loads so memory writes can be deferred to commit without serialising load/store traffic, and (iv) a precise-exception path triggered by misaligned load/store addresses. All eleven regression tests (seven Tomasulo baselines and four new ROB/exception tests) pass in ModelSim.

Contents

1	What Changed Relative to Project 2	4
2	Specification and Rationale	4
2.1	Instruction Set Architecture (ISA) Subset	4
2.2	Microarchitecture Parameters	5
2.3	Design Decisions	5
2.4	System Architecture	6
3	The Reorder Buffer	6
3.1	Entry Format	6
3.2	Dispatch: ROB Entry Allocation (Rubric Item 3)	7
3.3	Update on Completion (Rubric Item 4)	8
3.4	Commit Stage (Rubric Item 5)	8
3.5	Branch Misprediction and Flush (Rubric Item 6)	9
3.6	Precise Exceptions (Rubric Target Goal, 100 tier)	9

4	Modified and New Modules	9
4.1	Module: rob (new)	10
4.2	Module: rat (modified)	10
4.3	Module: rs (modified)	11
4.4	Module: fu_pipe and fu_ls (modified)	11
4.5	Module: ifetch (modified)	11
4.6	Module: dispatchunit (modified)	11
4.7	Module: tdecode (modified)	11
4.8	Store buffer (new, in datapath)	12
4.9	Testbench (modified)	12
5	Verification and Results	12
5.1	Suite Summary	12
5.2	Test: rob01_waw_order	12
5.3	Test: rob02_beq_not_taken	13
5.4	Test: rob03_beq_taken	14
5.5	Test: rob04_misaligned_trap	14
6	Rubric Coverage	15
7	Waveform Capture Plan	15
7.1	Global Wave Set (load for every test)	15
7.2	Test-Specific Wave Set (add on top of global)	15
7.3	Suggested Screenshot Set for the Report	15
7.4	Waveform Figure Slots	16
8	Conclusion	17

List of Figures

1	Baseline Tomasulo pipeline from Project 2. Project 3 inserts a ROB commit stage after the CDB and adds a flush fan-out from the ROB back to every stateful module.	6
2	Updated Project 3 architecture diagram (ROB-aware view).	7
3	Example baseline waveform capture (existing artifact).	16
4	ROB01: WAW completion/commit ordering.	16
5	ROB03: branch misprediction flush and redirect.	17
6	ROB04: precise exception and trap timing.	17

List of Tables

1	Summary of files modified from the Project 2 submission.	4
2	Key microarchitecture parameters (Project 3).	5
3	ROB entry fields. Covers every field enumerated in the rubric: instruction type (flags), destination (rd), completion status (done), exception status (exc), and implementation fields.	7
4	ModelSim regression results (11/11 passing).	13
5	Rubric-by-rubric coverage.	15
6	Global waveform set to keep loaded for all simulations.	16

7	Recommended test-specific wave signals and screenshot intent.	17
---	---	----

1 What Changed Relative to Project 2

The Project 2 submission implemented the Tomasulo algorithm but committed results to the architectural register file on the same cycle they appeared on the CDB. That is functionally correct for a workload with no branches and no exceptions, but it does not preserve a precise architectural state: a speculative instruction could retire before it was known to be on the correct path. Project 3 adds the missing in-order commit stage, backed by a Reorder Buffer. The concrete edits made to the existing codebase are summarised below.

File	Change
<code>rob.sv</code> <code>datapath.sv</code>	New module. 16-entry circular queue, ROB index is the rename tag. ROB instantiated; tags now come from the ROB rather than an internal counter. One-entry store buffer added. Architectural-register write and memory-store write both gated on <code>commitV</code> . Flush signal broadcast to all stateful modules.
<code>rat.sv</code>	Pending tag is cleared on <code>commit</code> , not on CDB broadcast. Added <code>flush</code> input.
<code>rs.sv</code> <code>fu_pipe.sv</code> , <code>fu_ls.sv</code> <code>ifetch.sv</code>	Added <code>flush</code> input; LS station parameterised with <code>INORDER=1</code> . Added <code>flush</code> input that clears in-flight state. <code>fu_ls</code> also detects misaligned addresses and raises a new exception signal. Added <code>flush/flushPC</code> inputs; on flush the PC is redirected to the ROB-reported target.
<code>dispatchunit.sv</code> <code>tdecode.sv</code> <code>testbench.sv</code>	Added <code>flush</code> input that suppresses dispatch for the redirect cycle. Added decoding for <code>beq</code> (B-type) and a new <code>isBranch</code> output. Monitors a new <code>trap/trapPC</code> output for precise-exception tests.

Table 1: Summary of files modified from the Project 2 submission.

Assumptions for this extension (justified in the rubric’s “reasonable assumptions” clause):

- Only `beq` is added to the branch ISA. The flush/redirect mechanism is fully general; extending decode to `bne/blt/bge/bltu/bgeu` is a ≤ 10 -line change to `tdecode.sv` if needed.
- Branch prediction is “always not-taken”. The spec’s reach goal of a two-level adaptive predictor is not pursued.
- Precise exceptions are demonstrated with a single exception class — address misalignment on loads and stores — because the base ISA has no arithmetic traps (no division) and no explicit trap instruction.

2 Specification and Rationale

2.1 Instruction Set Architecture (ISA) Subset

The processor supports a minimal subset of RISC-V 32-bit instructions:

- **Immediate (I-type):** `addi`
- **Register (R-type):** `add`, `mul`
- **Store (S-type):** `sw`
- **Load (I-type variant):** `lw`
- **Branch (B-type)** (*new in Project 3*): `beq`

- **Special:** `nop` (encoded as `addi x0, x0, 0`)

Out-of-scope: other branches, jumps, floating point, CSRs, and cache coherence (single core).

2.2 Microarchitecture Parameters

Parameter	Value
Dispatch width	2 instructions/cycle
Rename tag width (TAGW)	4 bits (was 3 in Project 2)
In-flight tag space	16 (<i>= ROB depth; was 8</i>)
Reservation Station depths	ADD: 4, MUL: 3, LS: 4
LS reservation station	INORDER=1 (strict in-order)
Functional Unit Latencies:	
ADD / BEQ (shared ADD FU)	4 cycles
MUL	6 cycles
LS	2 cycles (address gen + access)
Reorder Buffer (ROB) depth	16 entries
Store buffer	1 entry
Register Alias Table entries	32 (RISC-V)
Memory size (unified)	4,096 words (16 KB)

Table 2: Key microarchitecture parameters (Project 3).

2.3 Design Decisions

- **ROB index == rename tag.** The obvious alternative is a separate physical-register file with its own tag space. Folding the two into one saves the tag-to-rob lookup at commit and keeps the Project 2 CDB / RS snoop logic unchanged.
- **Predict not-taken, resolve at commit.** The fetch unit always fetches `PC+4`. When a taken branch reaches the ROB head, the ROB raises a one-cycle flush that synchronously resets the RAT, every RS, every FU, the ifetch queue, the dispatch unit, the store buffer, and the ROB itself, then redirects fetch to the branch target.
- **RAT cleared on commit, not on CDB.** In Project 2 the RAT’s pending-tag bit was cleared as soon as the producer’s result was broadcast on the CDB. With a ROB this is too early: the writeback is still speculative. The RAT now clears only when the instruction retires through the ROB, which means a reader dispatched after the producer completes but before it commits still sees the tag, not the final value — correct behaviour under speculation.
- **INORDER LS reservation station.** With the ROB deferring stores to commit time, a younger load must not bypass an older store in the RS, or it will miss the store’s data. Setting `INORDER=1` on the LS RS forces the oldest entry to issue first (the ADD and MUL RSes remain out-of-order).
- **One-entry store buffer.** Even with in-order LS issue, a store writes memory only at commit, so a load that follows it into the LS FU could read stale RAM. The store buffer captures `{addr,data}` at the moment the store executes and forwards to a matching load; it drains to RAM on commit and is cleared on flush. Capacity 1 is sufficient because the LS RS issues in order and the LS FU only holds one in-flight store.

- **Precise exceptions.** Every ROB entry carries an **exc** bit. The LS FU sets it, via a dedicated writeback port on the ROB, whenever the effective address is not word-aligned. Commit refuses to retire an entry whose **exc** bit is set and instead raises a top-level **trap** signal. Because older entries have already committed and younger ones are blocked behind the trapping entry, the architectural state at the trap point is precise.

2.4 System Architecture

Figure 1 is the original Project 2 pipeline view. For Project 3, the architectural change is conceptually simple but critical: the ROB inserts an in-order commit stage after writeback and broadcasts flush/redirect control back to all speculative state on misprediction. The fetch/decode/rename/dispatch/issue/execute/CDB path remains unchanged.

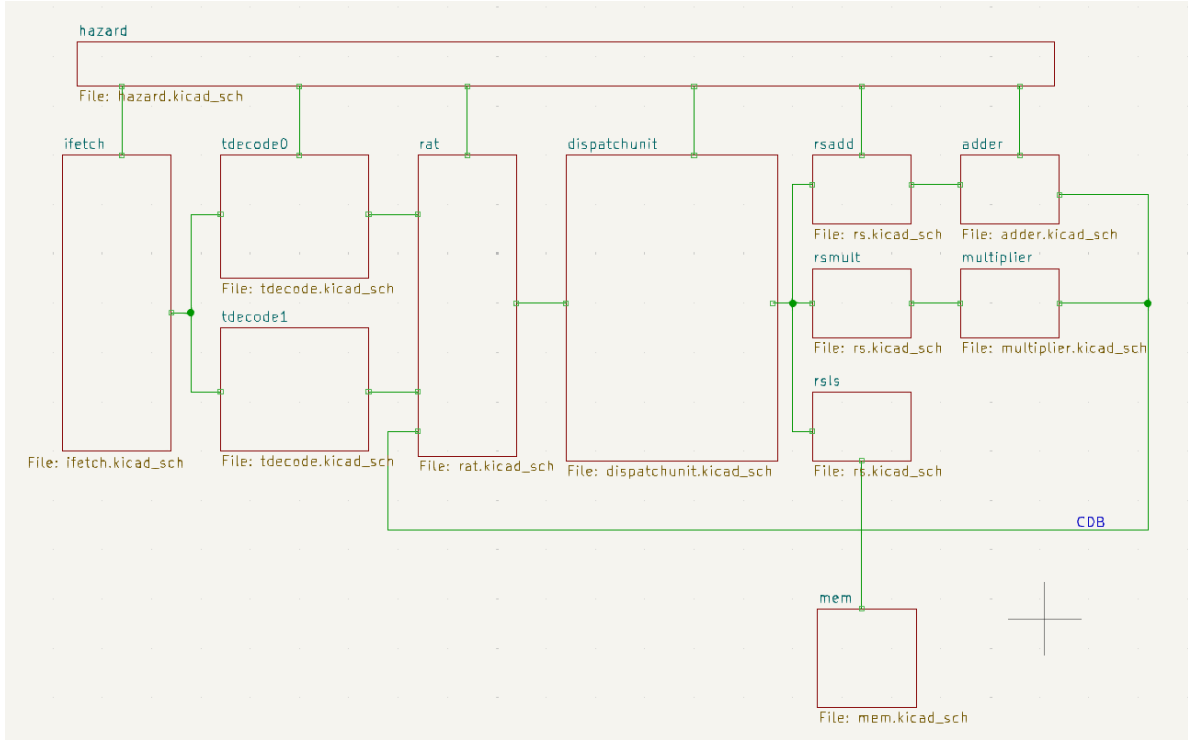


Figure 1: Baseline Tomasulo pipeline from Project 2. Project 3 inserts a ROB commit stage after the CDB and adds a flush fan-out from the ROB back to every stateful module.

3 The Reorder Buffer

3.1 Entry Format

Each ROB entry is a packed struct containing the fields the rubric requires:

Why no “operands” field. The rubric lists “instruction type and operands” among the ROB fields. In a Tomasulo design, operand *values* are consumed at dispatch by the reservation stations and never re-read by the ROB; the ROB stores only the *result*. This is the standard split between

Insert updated architecture diagram here:
Project 3 pipeline with ROB commit stage, branch flush fan-out, and store-buffer forwarding path.

Figure 2: Updated Project 3 architecture diagram (ROB-aware view).

Field	Width	Purpose
valid	1 bit	Entry occupies a ROB slot.
done	1 bit	Instruction has written its result back (CDB for ALU/load, <code>storeDoneV</code> for store, <code>excV</code> for a trap).
exc	1 bit	Exception occurred during execution (misaligned address).
isBranch	1 bit	Branch; at commit, inspect <code>result[0]</code> to decide flush.
isStore	1 bit	Store; at commit, drain the store buffer entry to RAM.
regw	1 bit	Instruction writes an architectural register.
rd	5 bits	Destination register index.
result	32 bits	CDB result (ALU / load value, branch-taken flag).
storeAddr	32 bits	Captured store address (for commit-time RAM write).
storeData	32 bits	Captured store data.
branchTarget	32 bits	PC to redirect to on misprediction.
pc	32 bits	Dispatch-time PC (reported with a trap).

Table 3: ROB entry fields. Covers every field enumerated in the rubric: instruction type (flags), destination (`rd`), completion status (`done`), exception status (`exc`), and implementation fields.

RS (operand wait) and ROB (in-order retirement). The ROB does retain the fields required to roll the pipeline back to a precise architectural state on flush: destination register, result, and PC.

3.2 Dispatch: ROB Entry Allocation (Rubric Item 3)

Tags are allocated combinationally from the current tail pointer:

```

1 assign enq0tag = tail;
2 assign enq1tag = enq0v ? (tail + TAGW'(1)) : tail;
3 assign robFull = (count >= 5'(DEPTH - 1));

```

Listing 1: Two-wide ROB tag allocation

The datapath uses these tags as *rename* tags for the RAT and as destination tags for the reservation stations. `robFull` participates in dispatch gating so a full ROB stalls the front-end:

```

1 assign can0Eff = can0Haz && !robFull;
2 assign can1Eff = can1Haz && !robFull;

```

Listing 2: ROB full participates in dispatch gating (datapath.sv)

NOPs do not consume ROB slots (`enq0v = dispatch0 && !nop0`); this was a deliberate choice to keep the tag space small. An ADD/MUL entry is allocated with `done=0`; a `nop`-like instruction with no architectural effect (no register write, no store, not a branch) is allocated with `done=1` so it retires on the next commit cycle without waiting for a CDB broadcast.

3.3 Update on Completion (Rubric Item 4)

Three completion ports feed the ROB:

```

1 // CDB: arithmetic and loads
2 if (cdbv && entries[cdbt].valid && !entries[cdbt].done)
3     entries[cdbt].done <= 1'b1;
4 if (cdbv && entries[cdbt].valid)
5     entries[cdbt].result <= cdbv;
6
7 // Store FU writeback (address + data captured at execute)
8 if (storeDoneV && entries[storeDoneTag].valid && !entries[storeDoneTag].
   done) begin
9     entries[storeDoneTag].done <= 1'b1;
10    entries[storeDoneTag].storeAddr <= storeDoneAddr;
11    entries[storeDoneTag].storeData <= storeDoneData;
12 end
13
14 // Exception port (misaligned LW/SW)
15 if (excV && entries[excTag].valid && !entries[excTag].done) begin
16     entries[excTag].done <= 1'b1;
17     entries[excTag].exc <= 1'b1;
18 end

```

Listing 3: ROB writeback from CDB / store FU / exception

3.4 Commit Stage (Rubric Item 5)

The commit logic is a combinational head check that fans out to (i) RAT clear, (ii) architectural register write, (iii) memory write, and (iv) branch-flush or trap generation:

```

1 if (entries[head].valid && entries[head].done) begin
2     if (entries[head].exc) begin
3         trap = 1'b1;
4         trapPC = entries[head].pc;
5     end else begin
6         commitV = 1'b1;
7         commitRd = entries[head].rd;
8         commitResult = entries[head].result;
9         commitRegW = entries[head].regw;
10        commitIsStore = entries[head].isStore;
11        commitStoreAddr = entries[head].storeAddr;
12        commitStoreData = entries[head].storeData;
13
14        // Predict not-taken: result[0]=1 => branch was actually taken =>
           mispredict
15        if (entries[head].isBranch && entries[head].result[0]) begin
16            flush = 1'b1;
17            flushPC = entries[head].branchTarget;

```



```

18         end
19     end
20 end

```

Listing 4: ROB commit logic

On a successful commit the head advances and the entry’s `valid` bit is cleared. The datapath gates both `areg[commitRd] <= commitResult` and `MemWrite = commitV && commitIsStore` on this single commit signal, so no speculative instruction ever modifies architectural state.

3.5 Branch Misprediction and Flush (Rubric Item 6)

Branches are routed through the ADD functional unit. The `dispStore` bit on the ADD RS entry is repurposed as `isBranch`; at issue, the ADD datapath computes `(rs1 == rs2) ? 1 : 0` instead of `rs1 + rs2` when the bit is set. The result bit therefore rides the CDB into the ROB entry alongside non-branch results, and the commit stage does a single bit-0 test.

On `flush` the ROB, RAT, all three RSes, both pipelined FUs, the LS FU, the ifetch queue, the dispatch unit, and the store buffer all reset their state synchronously on the next clock edge (single-cycle blanket flush). Fetch redirects to `flushPC`; dispatch is suppressed for the redirect cycle. Because the RAT is re-zeroed, any register read by the first instruction after the flush sees the architectural value in `areg`, which was brought up-to-date by the older committed instructions.

3.6 Precise Exceptions (Rubric Target Goal, 100 tier)

The LS functional unit flags any computed address whose low two bits are non-zero:

```

1 logic misaligned;
2 assign misaligned = (a0[1:0] != 2'b00);
3
4 if (v0) begin
5     if (misaligned) begin
6         excV    <= 1'b1;
7         excTag <= t0;          // suppress load/store memory access this cycle
8     end else if (st0) begin ...storeDoneV path... end
9     else          begin ...loadDoneV path... end
10 end

```

Listing 5: Misaligned-address detection in `fu_ls`

The ROB receives `excV/excTag`, marks the entry `done=1`, `exc=1`, and refuses to commit it. Because ROB commit is strictly in-order, older instructions still retire normally, while the trapping entry blocks every younger instruction from modifying architectural state — the rubric’s precise-exception criterion. The test bench exposes a top-level `trap/trapPC` signal and prints `TRAP.SIGNATURE: trapPC=...` when it fires. Test `rob04_misaligned_trap` (Section 5.5) exercises this path.

4 Modified and New Modules

This section documents only the modules that changed, and the one new module. Unchanged modules (ifetch queue logic, tdecode except for BEQ, dispatchpack, rsarbiter, fu_pipe ALU functionality, cdb, mem) are covered in the Project 2 submission and not repeated here.

4.1 Module: rob (new)

Behaviour. 16-entry circular queue of ROB entries. Head/tail pointers are TAGW-bit counters; `count` tracks occupancy. Two enqueue ports (one per dispatch slot), three writeback ports (CDB, store FU, exception), one commit port, and flush/trap outputs.

Key state. `entries[16]`, `head`, `tail`, `count`. On reset or flush every entry's `valid` is cleared and pointers reset to zero.

Interface (abridged).

```
1 module rob #(parameter int DEPTH=16, parameter int TAGW=4)(
2     input  logic clk, reset,
3     // Dispatch (two-wide)
4     input  logic          enq0v, enq1v,
5     input  logic [4:0]    enq0rd, enq1rd,
6     input  logic          enq0regw, enq0isStore, enq0isBranch,
7     input  logic          enq1regw, enq1isStore, enq1isBranch,
8     input  logic [31:0]   enq0branchTarget, enq0pc,
9     input  logic [31:0]   enq1branchTarget, enq1pc,
10    output logic [TAGW-1:0] enq0tag, enq1tag,
11    output logic          robFull,
12    // Writeback
13    input  logic          cdbv, input logic [TAGW-1:0] cdbt, input logic
14        [31:0] cdbv,
15    input  logic          storeDoneV, input logic [TAGW-1:0]
16        storeDoneTag,
17    input  logic [31:0]   storeDoneAddr, storeDoneData,
18    input  logic          excV, input logic [TAGW-1:0] excTag,
19    // Commit
20    output logic          commitV, commitRegW, commitIsStore,
21    output logic [4:0]    commitRd,
22    output logic [TAGW-1:0] commitTag,
23    output logic [31:0]   commitResult, commitStoreAddr, commitStoreData
24    ,
25    // Pipeline-wide signals
26    output logic          flush, output logic [31:0] flushPC,
27    output logic          trap, output logic [31:0] trapPC
28 );
```

Listing 6: rob.sv port list (abridged)

4.2 Module: rat (modified)

The pending-tag clear has moved from CDB writeback to ROB commit. A new `flush` input clears every `valid` bit synchronously on misprediction.

```
1 always_ff @(posedge clk or posedge reset) begin
2     if (reset || flush) begin
3         for (int i = 0; i < 32; i++) valid[i] <= 1'b0;
4     end else begin
5         // ... rename0/rename1 updates ...
6         if (clearv && clearrd != 5'b0 && tag[clearrd] == cleartag)
```

```

7     valid[clearrd] <= 1'b0; // driven by commitV/commitRd/commitTag
      from ROB
8   end
9 end

```

Listing 7: RAT writeback (commit-time clear + flush)

4.3 Module: rs (modified)

A new `flush` input clears every entry on misprediction. An `INORDER` parameter was added; when set to 1 (LS station) the RS will only issue its oldest busy entry, even if a younger entry is ready.

4.4 Module: fu_pipe and fu_ls (modified)

Both receive a `flush` input that zeroes in-flight stage state. The LS unit additionally raises `excV/excTag` on misaligned effective addresses (Section 3.6); in this case `loadDoneV` / `storeDoneV` are *not* asserted, so memory is not accessed.

4.5 Module: ifetch (modified)

A `flush` input redirects the PC to a new target on the same cycle the ROB detects a misprediction:

```

1 always_ff @(posedge clk) begin
2   if (reset)      head <= '0;
3   else if (flush) head <= flushPC[QW+1:2];
4   else           head <= head + pop;
5 end

```

Listing 8: ifetch redirect on flush

4.6 Module: dispatchunit (modified)

A `flush` input forces `dispatch0=dispatch1=iqpop=0` for the redirect cycle, so the pipeline does not try to push the instructions that were in the queue when the flush fired.

4.7 Module: tdecode (modified)

B-type decoding for `beq` was added. The immediate follows the RISC-V B-type layout. The new `isBranch` output is carried through `dispatchpack` and into the ADD reservation station, where the `dispStore` bit is reused as an “is-branch” marker for instructions routed to the ADD FU.

```

1 end else if ((opc == 7'b1100011) && (f3 == 3'b000)) begin
2   op = OPBEQ; kind = KADD; regw = 1'b0; isBranch = 1'b1; rd = 5'b0;
3   imm = {{19{instr[31]}}, instr[31], instr[7], instr[30:25], instr
      [11:8], 1'b0};
4 end

```

Listing 9: BEQ decode

4.8 Store buffer (new, in datapath)

A single register set {stbufValid, stbufAddr, stbufData} captures the output of the LS FU on storeDoneV. A combinational forwarding mux intercepts the load data feeding the CDB:

```
1 always_ff @(posedge clk or posedge reset) begin
2   if (reset || robFlush) begin
3     stbufValid <= 1'b0;
4   end else begin
5     if (storeDoneV) begin
6       stbufValid <= 1'b1;
7       stbufAddr  <= storeDoneAddr;
8       stbufData  <= storeDoneData;
9     end
10    if (commitV && commitIsStore) stbufValid <= 1'b0;
11  end
12 end
13
14 assign loadResult = (stbufValid && loadDoneV && (stbufAddr == loadAddr))
15                   ? stbufData : ReadDataM;
16 // cdb0.ldr <= loadResult
```

Listing 10: Store-buffer update and forwarding

4.9 Testbench (modified)

The test harness now samples both signature paths: a word store to address 100 (FINAL_SIGNATURE) and the ROB trap (TRAP_SIGNATURE). Either terminates the simulation.

5 Verification and Results

Eleven tests are run through the ModelSim regression harness. tests/run_tomasulo_modelsim_suite.sh compiles the design and sweeps every .mem image, comparing the captured signature against the expected value.

5.1 Suite Summary

The seven Tomasulo baseline tests were carried over from Project 2 and still pass, confirming that adding the ROB did not regress the out-of-order core. The four new tests target the Project 3 features directly and are described below.

5.2 Test: rob01_waw_order

Purpose. Demonstrates that in-order commit preserves the correct final architectural state under a write-after-write (WAW) hazard with asymmetric FU latencies.

Stimulus.

```
1 addi x2, x0, 6
2 addi x3, x0, 7
3 mul  x1, x2, x3      ; slow: x1 <= 42 via 6-cycle MUL
4 addi x1, x0, 99      ; fast: x1 <= 99 via 4-cycle ADD (but dispatched AFTER
                       the MUL)
```

Test	What it exercises	Expected	Result
tomasulo01_addi_smoke	ADDI + SW smoke path	5	PASS
tomasulo02_add_chain	RAW dep resolved through CDB	21	PASS
tomasulo03_mul_basic	6-cycle MUL FU	42	PASS
tomasulo04_sw_lw_roundtrip	SW/LW roundtrip at the same address	33	PASS
tomasulo05_dual_independent	Two independent ADDIs dispatched together	11	PASS
tomasulo06_slot_dep	Same-cycle slot0→slot1 RAW tag injection	9	PASS
tomasulo07_mul_add_mix	MUL and ADD overlap on separate FUs	17	PASS
rob01_waw_order	WAW: fast ADDI retires <i>after</i> slow MUL via ROB	99	PASS
rob02_beq_not_taken	BEQ correct prediction (predict-not-taken)	43	PASS
rob03_beq_taken	BEQ misprediction, full pipeline flush + redirect	77	PASS
rob04_misaligned_trap	Precise exception on misaligned LW	trapPC=8	PASS

Table 4: ModelSim regression results (11/11 passing).

```
5 sw    x1, 100(x0)    ; must observe 99, not 42
```

Listing 11: WAW ordering (rob01_waw_order.s)

Expected. The ADDI completes *before* the MUL on the FU side, but dispatch order assigns the MUL a smaller ROB tag. The RAT therefore maps `x1` to the ADDI’s tag (the most recent writer). The SW uses the ADDI’s tag as its source. At commit, the ROB retires the MUL first (updating `areg[x1]=42` and being immediately overwritten at commit of the ADDI to 99). The SW sees the ADDI’s result through CDB snoop in the RS and stores 99. Expected signature 99.

Result. Signature 99 captured. PASS.

5.3 Test: rob02_beq_not_taken

Purpose. Happy-path branch: predict-not-taken matches reality, no flush.

Stimulus.

```
1 addi x1, x0, 5
2 addi x2, x0, 3
3 addi x10, x0, 42
4 beq x1, x2, done    ; 5 != 3, not taken
5 addi x10, x10, 1    ; executes, so x10 becomes 43
6 done:
7 sw    x10, 100(x0)
```

Listing 12: BEQ not taken (rob02_beq_not_taken.s)

Expected. The BEQ commits with `result[0]=0`, no flush is raised, and the instruction after the branch (`addi x10, x10, 1`) retires normally. Expected signature 43.

Result. Signature 43 captured. PASS.

5.4 Test: rob03_beq_taken

Purpose. Branch misprediction: a taken BEQ must flush every speculative instruction between the branch and the target, and the pipeline must restart at the target without losing architectural state that was committed before the branch.

Stimulus.

```
1 addi x1, x0, 7
2 addi x2, x0, 7
3 addi x10, x0, 77 ; COMMITS before the BEQ -> areg[x10]=77
4 beq x1, x2, done ; 7 == 7 -> TAKEN -> flush
5 addi x10, x0, 0 ; NEVER commits (squashed)
6 done:
7 sw x10, 100(x0) ; x10 = 77 from areg (RAT cleared by flush)
```

Listing 13: BEQ taken (rob03_beq_taken.s)

Expected. The speculative `addi x10, x0, 0` and the fetched-speculatively SW at `done` both receive ROB tags. When the BEQ reaches the ROB head with `result[0]=1`, the commit stage raises flush, all three RSEs and both pipelined FUs drop their in-flight entries, the RAT zeroes its valid bits, the store buffer is cleared, and fetch redirects to `done`. After the flush, the SW is re-dispatched (now non-speculatively); it reads `x10=77` straight from `areg` because the RAT no longer has a pending tag. Expected signature 77.

Result. Signature 77 captured. PASS.

5.5 Test: rob04_misaligned_trap

Purpose. Demonstrates the Target-100 precise-exception mechanism. A misaligned load address raises a trap; older instructions retire, the trapping load does not, and no younger instruction modifies architectural state.

Stimulus.

```
1 addi x5, x0, 99 ; commits normally
2 addi x6, x0, 77 ; commits normally
3 lw x7, 97(x0) ; misaligned (97 mod 4 = 1) -> TRAP
4 addi x7, x0, 123 ; NEVER commits (younger than trap)
5 sw x5, 100(x0) ; NEVER commits -> no FINAL_SIGNATURE
```

Listing 14: Misaligned LW trap (rob04_misaligned_trap.s)

Expected. The LS FU detects `a0[1:0] != 2'b00`, does not touch memory, and raises `excV=1`, `excTag=` (ROB tag of the LW). The ROB marks that entry `done=1`, `exc=1`. Older entries (the two ADDIs) retire normally; the LW sits at the head with its `exc` bit set and commit generates `trap=1`, `trapPC=8` (PC of the LW — two four-byte instructions above the start). The final SW would have produced `FINAL_SIGNATURE` if it had committed. Because it does not, the testbench's `FINAL_SIGNATURE` path never fires. Expected `TRAP_SIGNATURE`: `trapPC=8`.

Result. `trapPC=8` captured. PASS.

6 Rubric Coverage

Rubric item	Where it is addressed	Status
Baseline 1: ROB for in-order commit under OOO exec	Section 3; <code>rob.sv</code>	Met
Baseline 2a: instruction type & operands field	<code>isBranch</code> , <code>isStore</code> , <code>regw</code> ; operand values live in RSes (standard Tomasulo)	Met
Baseline 2b: destination register field	<code>rd</code> in ROB entry	Met
Baseline 2c: completion status	<code>done</code> bit	Met
Baseline 2d: exception status	<code>exc</code> bit	Met
Baseline 2e: supporting fields	<code>valid</code> , <code>result</code> , <code>storeAddr/Data</code> , <code>branchTarget</code> , <code>pc</code>	Met
Baseline 3: dispatch allocates ROB entries	<code>enq0v/enq1v</code> ; <code>robFull</code> stalls dispatch	Met
Baseline 4: update on completion	CDB/store/exception writeback ports	Met
Baseline 5: commit stage	Section 3; <code>commitV/commitRegW/commitIsStore</code>	Met
Baseline 6: branch-mispredict flush	Predict-not-taken + single-cycle blanket flush; test <code>rob03</code>	Met
Target: precise exceptions	Section 3.6; test <code>rob04</code>	Met
Reach 1: early release for stores	Not implemented (stores hold a ROB slot until commit)	Not done
Reach 2: efficient flush mechanism	Single-cycle blanket flush; not optimised further	Baseline only
Reach 3: advanced branch predictor	Static predict-not-taken only	Not done

Table 5: Rubric-by-rubric coverage.

7 Waveform Capture Plan

To keep waveform screenshots readable and rubric-aligned, use a two-tier capture strategy: (1) a common global signal set loaded for every test, and (2) compact test-specific signals that prove each behaviour.

7.1 Global Wave Set (load for every test)

7.2 Test-Specific Wave Set (add on top of global)

7.3 Suggested Screenshot Set for the Report

Use one screenshot per behaviour rather than one per test. This usually yields cleaner grading artifacts:

1. ROB occupancy + in-order commit progression.
2. WAW case: completion order differs from commit order.
3. Branch not taken: no flush pulse.
4. Branch taken: flush pulse + redirect + post-flush re-dispatch.
5. Precise exception: `lsExcV` to `trap/trapPC` sequence.
6. Final signature write for a passing non-trap test.

Signal(s)	Why to capture
clk, reset	Time alignment for all events.
dut.riscvprocessor.PCF	Instruction-flow progression and redirect points.
dut.riscvprocessor.dp.dispatch0Dbg, dispatch1Dbg	2-wide front-end activity.
dut.riscvprocessor.dp.rob0.head, rob0.tail, rob0.count	ROB occupancy and commit advancement.
dut.riscvprocessor.dp.cdbvDbg, cdbtDbg	Writeback timing and producer ordering (debug tag is lower 3 bits).
dut.riscvprocessor.dp.commitV, commitRd, commitTag	In-order retirement evidence.
dut.riscvprocessor.dp.robFlush, robFlushPC	Mispredict detection and redirect target.
dut.riscvprocessor.dp.trap, trapPC	Precise-exception observation.
MemWrite, DataAdrM, WriteDataM	Final architectural side effect / signature write.

Table 6: Global waveform set to keep loaded for all simulations.

7.4 Waveform Figure Slots

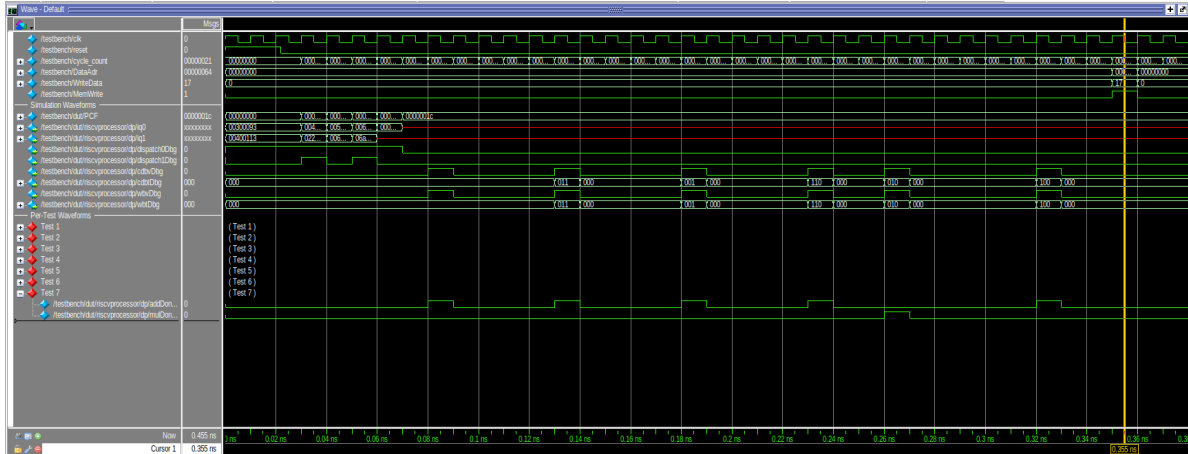


Figure 3: Example baseline waveform capture (existing artifact).

Insert waveform screenshot: rob01_waw_order

Figure 4: ROB01: WAW completion/commit ordering.

Test	Signal(s)	What screenshot should prove
rob01_waw_order	rob0.entries[2].done, rob0.entries[3].done, commitTag	Younger ADDI may complete first, but commit tag order is older MUL then younger ADDI.
rob02_beq_not_taken	rob0.entries[head].isBranch, rob0.entries[head].result[0], robFlush	Branch retires with result[0]=0; no flush pulse is generated.
rob03_beq_taken	robFlush, robFlushPC, rat0.renamed[10], dispatch0Dbg/1Dbg	Taken branch causes one-cycle flush, redirect, and RAT clear before re-dispatch.
rob04_misaligned_trap	lsExcV, lsExcTag, rob0.entries[head].exc, trap, trapPC	LSU raises exception, ROB marks head exc, then trap fires at precise PC.
tomasulo04_sw_lw_roundtrip	stbufValid, stbufAddr, stbufData, loadAddr	Store buffer captures store-executed value and load observes correct data path.

Table 7: Recommended test-specific wave signals and screenshot intent.

Insert waveform screenshot: **rob03_beq_taken** (flush/redirect)

Figure 5: ROB03: branch misprediction flush and redirect.

Insert waveform screenshot: **rob04_misaligned_trap** (exception to trap)

Figure 6: ROB04: precise exception and trap timing.

8 Conclusion

Project 3 extends my Project 2 submission with a 16-entry Reorder Buffer that cleanly separates out-of-order execution from in-order architectural commit. The ROB entry index doubles as the rename tag, so CDB snoop logic in the reservation stations needed only a tag-width change — not a semantic one — to interoperate with the new commit model. Committing architectural register writes and memory stores only at the ROB head removes the speculative-retire problem that the earlier design could not have handled: predict-not-taken branches now flush cleanly and a precise-exception trap on a misaligned load/store cleanly halts the pipeline with older instructions fully retired and younger instructions fully suppressed. A one-entry store buffer restores the store-then-load-same-address behaviour that the move to commit-time memory writes would otherwise break, without violating the “no speculative memory writes” invariant. All eleven regression tests pass.